УЧЕБНИК ПО PHP

Для знающих HTML, без JavaScript

Этот учебник рассчитан на читателя, который уже уверенно пишет HTML-страницы, но не знаком с JavaScript и серверной разработкой. Мы начинаем с минимума: как запустить PHP локально, как встроить код в HTML-страницу, — и доходим до работы с базой данных MySQL через PDO, основ ООП и базовых приёмов безопасности.

Все примеры написаны на PHP 8.x. Код можно сразу проверять: сохраните файл с расширением .php и откройте через локальный сервер PHP. Главы построены по нарастающей сложности: каждая следующая опирается на материал предыдущих.

ОГЛАВЛЕНИЕ

- Глава 1. Введение в PHP
- Глава 2. Синтаксис и переменные
- Глава 3. Функции
- Глава 4. Массивы
- Глава 5. Формы и суперглобалы
- Глава 6. Работа с файлами
- Глава 7. Сессии и куки
- Глава 8. MySQL и PDO
- Глава 9. Основы ООП
- Глава 10. Безопасность
- Глава 11. Практические задачи
- Глава 12. Итоги и справочник функций

ГЛАВА 1. ВВЕДЕНИЕ В PHP

1.1. Что такое PHP

PHP (рекурсивный акроним «PHP: Hypertext Preprocessor») — это серверный язык программирования, созданный специально для веб-разработки. В отличие от HTML, который браузер пользователя читает и отображает, PHP-код выполняется на сервере, а пользователю отправляется уже готовый HTML.

Простейшая аналогия: HTML — это черновик документа, который вы отдаёте читателю как есть. PHP — это секретарь, который по вашим инструкциям собирает готовый документ: подставляет сегодняшнюю дату, вставляет список товаров из базы данных, формирует персональное приветствие с именем посетителя.

Важно понимать разницу между PHP и JavaScript. JavaScript выполняется в браузере пользователя, после того как страница загрузилась. PHP выполняется раньше — на сервере, ещё до того, как страница попадёт в браузер. Поэтому PHP может обращаться к базе данных, читать файлы на сервере, проверять пароли — всё это скрыто от пользователя.

1.2. Как запустить PHP локально

Чтобы проверить примеры из учебника, нужен установленный PHP. На macOS и большинстве Linux-дистрибутивов PHP уже есть или ставится через пакетный менеджер (apt, brew). На Windows проще всего использовать XAMPP или WSL.

Проверим, что PHP установлен:

```
php -v
# должно вывести что-то вроде:
# PHP 8.3.0 (cli) (built: ... )
```

PHP имеет встроенный веб-сервер для разработки. Перейдите в папку с вашим проектом и запустите:

```
cd ~/myproject
php -S localhost:8000
```

Теперь откройте в браузере адрес http://localhost:8000/ — и вы увидите файл index.php из этой папки, если он существует. Если в папке только index.html, PHP отдаст его как есть. Это самый простой способ начать работать без настройки Apache и Nginx.

1.3. Первая PHP-страница

PHP-код встраивается прямо в HTML с помощью специальных тегов <?php ... ?>. Всё, что внутри этих тегов, выполняется на сервере. Остальное отдаётся как обычный HTML.

```
&lt;!DOCTYPE html&gt;
&lt;html&gt;
  &lt;head&gt;&lt;title&gt;Первая страница&lt;/title&gt;&lt;/head&gt;
  &lt;body&gt;
    &lt;h1&gt;Привет, &lt;?php echo 'Мир'; ?&gt;!&lt;/h1&gt;
    &lt;p&gt;Сегодня &lt;?php echo date('d.m.Y'); ?&gt;.&lt;/p&gt;
  &lt;/body&gt;
&lt;/html&gt;
```

Сохраните этот код в файл index.php, перезапустите сервер, если нужно, и откройте страницу. В браузере вы увидите «Привет, Мир!» и текущую дату. Если посмотреть исходный код страницы (Ctrl+U), PHP-тегов там не будет — только результат их работы.

Короткая запись <?php echo $x; ?> настолько частая, что для неё существует сокращение <?= $x ?>. Это особенно удобно внутри HTML-шаблонов:

```
&lt;h1&gt;Привет, &lt;?= $name ?&gt;!&lt;/h1&gt;
```

ГЛАВА 2. СИНТАКСИС И ПЕРЕМЕННЫЕ

2.1. Переменные

Переменная в PHP начинается со знака доллара. Имя может содержать буквы, цифры и подчёркивание, но не должно начинаться с цифры. PHP чувствителен к регистру: $name и $Name — разные переменные.

```
&lt;?php
$name = 'Анна';
$age  = 28;
$is_admin = true;

echo $name;  // Анна
?&gt;
```

Переменные в PHP не требуют объявления типа: интерпретатор сам определяет тип по значению. Это называется динамической типизацией. Однако PHP 8.x

поддерживает необязательные подсказки типов в функциях — об этом в главе 3.

2.2. Типы данных

Основные типы PHP:

- int — целое число: 42, -7, 0
- float — число с плавающей точкой: 3.14, -0.5
- string — строка: 'текст', "другой текст"
- bool — логическое значение: true или false
- null — отсутствие значения
- array — массив (глава 4)
- object — объект класса (глава 9)

Проверить тип переменной можно функцией gettype() или оператором instanceof для объектов:

```
$x = 42;
echo gettype($x);  // integer

$y = 'привет';
echo gettype($y);  // string
```

Строки можно записывать в одинарных или двойных кавычках. Разница: в двойных кавычках PHP раскрывает переменные и специальные символы (\n, \t), а в одинарных — нет.

```
$name = 'Мир';
echo "Привет, $name!";  // Привет, Мир!
echo 'Привет, $name!';    // Привет, $name!
```

2.3. Операторы

Арифметические операторы стандартные: + - * / % (остаток от деления), ** (возведение в степень). Оператор точки . объединяет строки (конкатенация):

```
$a = 10;
$b = 3;
echo $a + $b;   // 13
echo $a % $b;   // 1
echo $a ** $b;  // 1000

$first = 'Иван';
$last  = 'Петров';
echo $first . ' ' . $last;  // Иван Петров
```

Операторы сравнения: == (равно по значению), === (строгое равенство — и значение, и тип), != и !==, < > <= >=. В PHP 8.x появился оператор spaceship <=>, который возвращает -1, 0 или 1.

```
var_dump(1 == '1');   // true (приведение типов)
var_dump(1 === '1'); // false (строгое)
var_dump(1 &lt;=&gt; 2);   // -1
var_dump(2 &lt;=&gt; 2);   // 0
var_dump(3 &lt;=&gt; 2);   // 1
```

Логические операторы: && (и), || (или), ! (не). Альтернативные слова and, or, xor — имеют более низкий приоритет, поэтому в сложных выражениях их лучше избегать.

2.4. Условия: if / elseif / else

Условный оператор выглядит так же, как в большинстве Си-подобных языков:

```
$age = 17;

if ($age &gt;= 18) {
    echo 'Доступ разрешён';
} elseif ($age &gt;= 16) {
    echo 'Доступ с родителем';
} else {
    echo 'Доступ запрещён';
}
```

В PHP 8.x можно использовать сокращённый синтаксис для вывода условия прямо в HTML:

```
&lt;?php if ($is_admin): ?&gt;
  &lt;a href="/admin"&gt;Админ-панель&lt;/a&gt;
&lt;?php endif; ?&gt;
```

2.5. match вместо switch

Конструкция switch остаётся рабочей, но в PHP 8.x появилась более удобная — match. Она возвращает значение и использует строгое сравнение:

```
$status = 'paid';

$label = match ($status) {
    'new'   =&gt; 'Новый заказ',
    'paid'  =&gt; 'Оплачен',
    'ship'  =&gt; 'Отправлен',
    default =&gt; 'Неизвестный статус',
};

echo $label;  // Оплачен
```

Преимущества match: короткий синтаксис, возврат значения, строгое === вместо ==, и TypeError, если ни одна ветка не подошла и нет default.

2.6. Циклы

Цикл while повторяет код, пока условие истинно:

```
$i = 1;
while ($i &lt;= 5) {
    echo $i . ' ';
    $i++;
}
// 1 2 3 4 5
```

Цикл for — это while с инициализацией и шагом в одной строке:

```
for ($i = 1; $i &lt;= 5; $i++) {
    echo $i . ' ';
}
```

Самый полезный цикл для веб-разработки — foreach. Он перебирает массивы и будет подробно разобран в главе 4. Базовая форма:

```
$colors = ['красный', 'зелёный', 'синий'];

foreach ($colors as $color) {
    echo "&lt;li&gt;$color&lt;/li&gt;\n";
}
```

ГЛАВА 3. ФУНКЦИИ

3.1. Объявление функции

Функции позволяют выделить повторяющийся код и вызывать его по имени. Функция объявляется ключевым словом function:

```
&lt;?php
function greet($name) {
    return "Привет, $name!";
}

echo greet('Анна');  // Привет, Анна!
echo greet('Иван');  // Привет, Иван!
```

Если функция не возвращает значение явно, она возвращает null. Возврат multiple values через массив — распространённый приём:

```
function minmax(array $nums): array {
    return [min($nums), max($nums)];
}

[$min, $max] = minmax([3, 7, 1, 9, 4]);
echo "$min ... $max";  // 1 ... 9
```

3.2. Аргументы по умолчанию

Параметры могут иметь значения по умолчанию. Если при вызове аргумент не передан, используется значение по умолчанию:

```
function greet($name, $greeting = 'Привет') {
    return "$greeting, $name!";
}

echo greet('Анна');           // Привет, Анна!
echo greet('Анна', 'Здравствуй');
// Здравствуй, Анна!
```

Параметры со значениями по умолчанию должны идти в конце списка аргументов. Иначе PHP не сможет понять, какой именно аргумент вы пропустили при вызове.

3.3. Именованные аргументы (PHP 8)

PHP 8.x добавил именованные аргументы. Теперь можно передавать параметры в любом порядке, указывая их имена:

```
function make_box($width, $height, $color, $border) {
    // ...
}

make_box(
    color: 'red',
    width: 200,
    height: 100,
    border: true,
);
```

Это особенно полезно для функций с большим числом необязательных параметров — больше не приходится передавать все значения по умолчанию, чтобы добраться до последнего.

3.4. Типизация параметров и возврата

PHP 8.x поддерживает строгую типизацию через declare(strict_types=1) в начале файла и явные подсказки типов:

```
&lt;?php
declare(strict_types=1);

function add(int $a, int $b): int {
    return $a + $b;
}

echo add(2, 3);  // 5
// echo add('2', '3');  // TypeError в строгом режиме
```

Тип возвращаемого значения указывается после двоеточия. Возможные типы: int, float, string, bool, array, callable, имена классов, ?int (int или null), void (ничего не возвращает), mixed (любой тип).

3.5. Область видимости

Переменные, созданные внутри функции, существуют только в ней. Переменные снаружи функции внутри неё недоступны, если не использовать global (что почти всегда плохая идея) или не передать через аргументы:

```
$counter = 0;

function increment() {
    global $counter;  // НЕ РЕКОМЕНДУЕТСЯ
    $counter++;
}

// Лучше — передать по ссылке или вернуть:
function inc(int $n): int {
    return $n + 1;
}
$counter = inc($counter);
```

Главное правило: функция должна быть самодостаточной — получать данные через аргументы и отдавать через return. Тогда её легко тестировать и переиспользовать.

3.6. include и require

PHP позволяет разбивать код на несколько файлов и подключать их. Есть четыре инструкции:

- include 'file.php' — подключает, если файла нет — warning
- require 'file.php' — подключает, если файла нет — fatal error
- include_once — как include, но подключает один раз
- require_once — как require, но один раз

Разница между include и require — только в реакции на отсутствие файла. require_once — самый безопасный выбор для подключения библиотек и конфигов: файл подключится ровно один раз, даже если на него ссылаются из нескольких мест.

```
&lt;?php
require_once 'config.php';
require_once 'functions.php';

// Теперь $config и функции доступны
echo $config['site_name'];
```

ГЛАВА 4. МАССИВЫ

4.1. Индексные массивы

Массив в PHP — это упорядоченный набор значений. Индексный массив имеет числовые ключи, начиная с 0:

```
$colors = ['красный', 'зелёный', 'синий'];

echo $colors[0];  // красный
echo $colors[1];  // зелёный
echo count($colors);  // 3

$colors[] = 'жёлтый';  // добавить в конец
print_r($colors);
```

4.2. Ассоциативные массивы

Ассоциативный массив использует строковые ключи. Это аналог словаря в Python или объекта в JavaScript:

```
$user = [
    'name' =&gt; 'Анна',
    'age'  =&gt; 28,
    'city' =&gt; 'Москва',
];

echo $user['name'];  // Анна
echo $user['age'];   // 28

$user['email'] = 'anna@example.com';  // добавить
unset($user['city']);  // удалить ключ
```

Ассоциативные массивы — самый частый способ представлять структурированные данные в PHP: конфиги, данные пользователя, результаты из базы данных.

4.3. Многомерные массивы

Значения массива могут быть сами массивами. Так строятся таблицы, списки объектов, деревья:

```
$users = [
    ['name' =&gt; 'Анна', 'age' =&gt; 28],
    ['name' =&gt; 'Иван', 'age' =&gt; 34],
    ['name' =&gt; 'Олег', 'age' =&gt; 19],
];

echo $users[1]['name'];  // Иван

foreach ($users as $u) {
    echo $u['name'] . ' — ' . $u['age'] . "\n";
}
```

4.4. foreach с ключом

Цикл foreach перебирает массивы. Полная форма получает и ключ, и значение:

```
$user = ['name' =&gt; 'Анна', 'age' =&gt; 28];

foreach ($user as $key =&gt; $value) {
    echo "$key: $value\n";
}
// name: Анна
// age: 28
```

Чтобы изменить исходный массив внутри foreach, добавьте & перед переменной значения:

```
$nums = [1, 2, 3];

foreach ($nums as &amp;$n) {
    $n *= 10;
}
unset($n);  // обязательно разорвать ссылку

print_r($nums);  // [10, 20, 30]
```

4.5. Полезные функции массивов

PHP имеет больше 80 встроенных функций для массивов. Самые ходовые:

- count($arr) — количество элементов
- array_keys($arr) — все ключи
- array_values($arr) — все значения
- in_array($x, $arr) — есть ли значение в массиве
- array_merge($a, $b) — объединить два массива
- array_map(fn, $arr) — применить функцию ко всем элементам
- array_filter($arr, fn) — оставить элементы, где fn вернула true

- sort($arr) — отсортировать по значению (in-place)
- usort($arr, fn) — сортировать по своей функции

```
$nums = [1, 2, 3, 4, 5];

// Удвоить все элементы
$doubled = array_map(fn($n) =&gt; $n * 2, $nums);
// [2, 4, 6, 8, 10]

// Оставить только чётные
$even = array_filter($nums, fn($n) =&gt; $n % 2 === 0);
// [2, 4]
```

Стрелочные функции fn($x) => ... появились в PHP 7.4 и удобны для коротких операций в array_map и array_filter. Они автоматически захватывают переменные из внешней области.

4.6. Spread-оператор и деструктуризация

Spread-оператор ... распаковывает массив. Удобно для объединения и передачи аргументов:

```
$a = [1, 2, 3];
$b = [4, 5, 6];
$c = [...$a, ...$b];  // [1, 2, 3, 4, 5, 6]

// Деструктуризация
[$first, $second] = $a;
echo $first;  // 1

// Для ассоциативных массивов (PHP 8.1+)
['name' =&gt; $name, 'age' =&gt; $age] = $user;
```

ГЛАВА 5. ФОРМЫ И СУПЕРГЛОБАЛЫ

5.1. Суперглобальные массивы

PHP автоматически собирает данные из разных источников в специальные суперглобальные массивы. Они доступны в любой точке кода — внутри функций и методов без слова global:

- $_GET — данные из URL (query string)
- $_POST — данные из тела POST-запроса (формы)
- $_REQUEST — объединение $_GET, $_POST, $_COOKIE

- $_SERVER — заголовки и метаданные запроса
- $_FILES — загруженные файлы
- $_COOKIE — куки
- $_SESSION — данные сессии

Самые нужные элементы $_SERVER:

- $_SERVER['REQUEST_METHOD'] — GET или POST
- $_SERVER['REMOTE_ADDR'] — IP клиента
- $_SERVER['HTTP_USER_AGENT'] — браузер клиента
- $_SERVER['REQUEST_URI'] — текущий путь URL

5.2. Простая форма

HTML-форма отправляет данные на сервер. Атрибут method определяет, как — GET (через URL) или POST (через тело запроса):

```
&lt;form method="post" action="handle.php"&gt;
  &lt;input type="text" name="username"
         placeholder="Ваше имя"&gt;
  &lt;button type="submit"&gt;Отправить&lt;/button&gt;
&lt;/form&gt;
```

Обработчик handle.php:

```
&lt;?php
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $name = $_POST['username'] ?? '';
    if ($name !== '') {
        echo "Привет, " . htmlspecialchars($name);
    } else {
        echo 'Введите имя';
    }
}
?&gt;
```

Здесь появилось сразу два важных приёма. Оператор ?? (null coalescing) возвращает значение слева, если оно не null, иначе — справа. Это безопасный способ обратиться к ключу массива: $_POST['username'] ?? '' не выдаст warning, если ключа нет.

Функция htmlspecialchars() экранирует специальные символы HTML. Если её не вызвать, пользователь сможет ввести <script> вместо имени и выполнить XSS-атаку. Подробнее об этом — в главе 10.

5.3. GET vs POST

GET — для чтения данных. Параметры видны в URL (?name=Anna&age=28), страницу можно заложить в закладки, ссылку можно отправить. Лимит длины URL — около 2000 символов.

POST — для изменения данных и отправки больших объёмов. Параметры в теле запроса, не видны в URL, не попадают в историю браузера. Используется для форм регистрации, входа, отправки сообщений, загрузки файлов.

Правило: если действие изменяет состояние на сервере (создаёт, обновляет, удаляет) — POST. Если просто фильтрует или показывает — GET. Это правило называется REST и соблюдается во всём вебе.

5.4. Чекбоксы и радиокнопки

Чекбоксы с одинаковым name плюс квадратные скобки отправляются как массив:

```
&lt;form method="post"&gt;
  &lt;label&gt;&lt;input type="checkbox" name="colors[]"
         value="red"&gt; Красный&lt;/label&gt;
  &lt;label&gt;&lt;input type="checkbox" name="colors[]"
         value="green"&gt; Зелёный&lt;/label&gt;
  &lt;label&gt;&lt;input type="checkbox" name="colors[]"
         value="blue"&gt; Синий&lt;/label&gt;
  &lt;button&gt;Отправить&lt;/button&gt;
&lt;/form&gt;
```

```
&lt;?php
$colors = $_POST['colors'] ?? [];
foreach ($colors as $c) {
    echo htmlspecialchars($c) . "&lt;br&gt;\n";
}
?&gt;
```

Радиокнопки (radio) — наоборот, всегда отдают одно значение, поэтому name у них одинаковый без скобок.

5.5. Валидация формы

Никогда не доверяйте данным из формы. Проверяйте всё: что обязательные поля заполнены, что email похож на email, что возраст — число, что выбранное значение попадает в список разрешённых.

```
&lt;?php
$errors = [];

$name  = trim($_POST['name'] ?? '');
$email = trim($_POST['email'] ?? '');
$age   = $_POST['age'] ?? '';

if ($name === '') {
    $errors[] = 'Имя обязательно';
}
if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
    $errors[] = 'Некорректный email';
}
if (!ctype_digit($age) || (int)$age &lt; 1) {
    $errors[] = 'Возраст должен быть положительным';
}

if ($errors) {
    foreach ($errors as $e) echo "$e&lt;br&gt;\n";
} else {
    echo 'Данные корректны';
}
?&gt;
```

Функция filter_var с фильтром FILTER_VALIDATE_EMAIL — самый надёжный способ проверить email без собственных регулярных выражений. Триммирование убирает случайные пробелы по краям.

ГЛАВА 6. РАБОТА С ФАЙЛАМИ

6.1. Простое чтение и запись

Самый простой способ прочитать файл целиком — file_get_contents. Записать — file_put_contents:

```
&lt;?php
// Чтение
$content = file_get_contents('data.txt');
echo $content;

// Запись (перезапишет файл)
file_put_contents('data.txt', 'новый текст');

// Дозапись в конец
file_put_contents('log.txt', "новая строка\n",
    FILE_APPEND | LOCK_EX);
?&gt;
```

Флаг FILE_APPEND добавляет данные в конец файла, не стирая существующие. LOCK_EX блокирует файл на время записи, чтобы параллельный процесс не записал одновременно и не испортил данные. Используйте их всегда при дозаписи.

6.2. file и построчное чтение

Функция file() читает файл и возвращает массив его строк. Удобно для обработки построчно:

```
$lines = file('log.txt', FILE_IGNORE_NEW_LINES);
foreach ($lines as $i =&gt; $line) {
    echo ($i + 1) . ': ' . $line . "\n";
}
```

Для больших файлов лучше fopen/fgets — читают по одной строке за раз, не загружая весь файл в память:

```
$fh = fopen('big.log', 'r');
while (($line = fgets($fh)) !== false) {
    if (str_contains($line, 'ERROR')) {
        echo $line;
    }
}
fclose($fh);
```

Функция str_contains появилась в PHP 8.0 и заменила длинные конструкции с strpos.

6.3. Загрузка файлов

HTML-форма для загрузки файлов обязательно должна иметь атрибут enctype="multipart/form-data" и метод POST:

```
&lt;form method="post" enctype="multipart/form-data"&gt;
  &lt;input type="file" name="avatar"&gt;
  &lt;button&gt;Загрузить&lt;/button&gt;
&lt;/form&gt;
```

PHP кладёт информацию о файле в суперглобал $_FILES:

```
$f = $_FILES['avatar'];
// $f — массив с ключами:
//   name     — оригинальное имя файла
//   tmp_name — временный путь на сервере
//   size     — размер в байтах
//   type     — MIME-тип из браузера (НЕ доверять!)
//   error    — код ошибки, UPLOAD_ERR_OK = 0
```

Файл нужно перенести из временной папки в постоянную — иначе он удалится после завершения скрипта:

```
if ($_FILES['avatar']['error'] === UPLOAD_ERR_OK) {
    $tmp  = $_FILES['avatar']['tmp_name'];
    $name = basename($_FILES['avatar']['name']);
    $dest = __DIR__ . '/uploads/' . $name;

    if (move_uploaded_file($tmp, $dest)) {
        echo 'Загружено: ' . $name;
    } else {
        echo 'Ошибка перемещения';
    }
}
```

6.4. Проверки загружаемого файла

Минимальный набор проверок при загрузке файла: размер, тип, расширение. Никогда не доверяйте $_FILES['...']['type'] — это значение присылает браузер и его легко подделать. Проверяйте MIME на сервере:

```
$allowed = ['jpg' =&gt; 'image/jpeg',
            'png' =&gt; 'image/png',
            'gif' =&gt; 'image/gif'];

$ext = strtolower(pathinfo($name, PATHINFO_EXTENSION));
if (!isset($allowed[$ext])) {
    die('Недопустимое расширение');
}

$mime = mime_content_type($tmp);
if ($allowed[$ext] !== $mime) {
    die('MIME не соответствует расширению');
}

if ($_FILES['avatar']['size'] &gt; 2 * 1024 * 1024) {
    die('Файл больше 2 МБ');
}
```

Функция pathinfo с флагом PATHINFO_EXTENSION достаёт расширение из имени файла. Функция mime_content_type определяет реальный MIME-тип по содержимому, а не по заявлению браузера.

ГЛАВА 7. СЕССИИ И КУКИ

7.1. Что такое куки и сессии

HTTP — протокол без состояния: каждый запрос ничего не знает о предыдущем. Чтобы «запомнить» пользователя между переходами по страницам, нужны куки и сессии.

Куки — это пары ключ=значение, которые сервер отправляет браузеру в HTTP-заголовке. Браузер сохраняет их и прикладывает к каждому следующему запросу. Куки хранятся на стороне клиента, ограничены по размеру (4 КБ) и доступны для подделки.

Сессия — это данные, хранящиеся на сервере в файлах (по умолчанию). Браузеру отправляется только один идентификатор сессии — PHPSESSID. Сам пользователь видит только идентификатор, а реальные данные (логин, корзина, права) лежат на сервере. Это безопаснее: даже если кто-то подделает идентификатор, ему всё равно нужен будет корректный PHPSESSID от реальной сессии.

7.2. Работа с куками

Установить куку можно функцией setcookie. Важно: она должна быть вызвана ДО любого вывода — до echo, до HTML, до пустых строк вне <?php ?>.

```
&lt;?php
setcookie('user', 'Anna', time() + 3600, '/');
echo 'Кука установлена';
?&gt;
```

Здесь time() + 3600 — срок жизни куки в секундах от текущего момента. Один час. Через час браузер сам её удалит. Параметр '/' — путь: кука доступна на всех страницах сайта.

Читать куки — через суперглобал $_COOKIE:

```
$user = $_COOKIE['user'] ?? 'гость';
echo "Привет, $user";
```

Удалить куку — установить её с прошедшим сроком:

```
setcookie('user', '', time() - 3600, '/');
unset($_COOKIE['user']);
```

7.3. Запуск сессии

Сессия начинается функцией session_start(). Она должна быть вызвана до любого вывода в каждом скрипте, где используются сессии:

```
&lt;?php
session_start();

$_SESSION['visits'] = ($_SESSION['visits'] ?? 0) + 1;

echo 'Вы посетили страницу '
   . $_SESSION['visits'] . ' раз.';
?&gt;
```

После session_start() суперглобал $_SESSION становится доступным. Записываете в него всё, что нужно сохранить между запросами: id пользователя, корзину, токены CSRF. PHP сам сериализует массив $_SESSION в файл на сервере в конце работы скрипта.

7.4. Авторизация на сессии

Типичный сценарий авторизации: при успешной проверке пароля сохраняем в сессию id пользователя. На каждой закрытой странице проверяем, есть ли этот id. Выход — уничтожение сессии.

```
&lt;?php
session_start();

// Вход (после проверки пароля)
if ($password_ok) {
    $_SESSION['user_id'] = $user['id'];
    header('Location: /dashboard.php');
    exit;
}
?&gt;
```

Проверка доступа в начале каждой закрытой страницы:

```
&lt;?php
session_start();
if (!isset($_SESSION['user_id'])) {
    header('Location: /login.php');
    exit;
}
$uid = $_SESSION['user_id'];
?&gt;
```

Выход:

```
session_start();
$_SESSION = [];          // очистить массив
session_destroy();       // удалить файл сессии
// Удалить куку PHPSESSID в браузере
setcookie(session_name(), '', time() - 3600, '/');
```

ГЛАВА 8. MySQL И PDO

8.1. Что такое PDO

PDO (PHP Data Objects) — это стандартный способ работы с базами данных в PHP. В отличие от старых функций mysqli, PDO поддерживает разные СУБД (MySQL, PostgreSQL, SQLite) через один интерфейс и обязательно использует подготовленные запросы, которые защищают от SQL-инъекций.

8.2. Подключение к базе

Подключение создаётся через создание объекта PDO с DSN — строкой, описывающей тип базы и параметры:

```
&lt;?php
$dsn = 'mysql:host=localhost;dbname=test;charset=utf8mb4';
$pdo = new PDO($dsn, 'user', 'pass', [
    PDO::ATTR_ERRMODE =&gt; PDO::ERRMODE_EXCEPTION,
    PDO::ATTR_DEFAULT_FETCH_MODE =&gt; PDO::FETCH_ASSOC,
]);
?&gt;
```

Здесь сразу задаются два важных режима: ERRMODE_EXCEPTION заставляет PDO бросать исключение при любой ошибке SQL (без него ошибки молча игнорируются), а FETCH_ASSOC делает так, что выборка возвращает ассоциативные массивы с ключами-именами колонок (по умолчанию PDO возвращает и индексы, и имена — двойной массив, что неудобно).

8.3. Подготовленные запросы

Главное правило: никогда не вставляйте данные пользователя прямо в SQL. Используйте подстановки — знаки вопроса или именованные :name — и передавайте значения отдельно через execute():

```
// Знаки вопроса
$stmt = $pdo-&gt;prepare(
    'SELECT * FROM users WHERE age &gt; ? AND city = ?'
);
$stmt-&gt;execute([18, 'Москва']);
$rows = $stmt-&gt;fetchAll();

// Именованные плейсхолдеры
$stmt = $pdo-&gt;prepare(
    'SELECT * FROM users WHERE email = :email'
);
$stmt-&gt;execute(['email' =&gt; $email]);
$user = $stmt-&gt;fetch();
```

PDO сам подставит значения и экранирует специальные символы. Это 100% защита от SQL-инъекций. Подготовленный запрос можно выполнить несколько раз с разными данными — сервер один раз строит план запроса и переиспользует его.

8.4. CRUD: create, read, update, delete

Создание записи (INSERT):

```
$stmt = $pdo-&gt;prepare(
    'INSERT INTO users (name, email, age)
     VALUES (:name, :email, :age)'
);
$stmt-&gt;execute([
    'name'  =&gt; 'Анна',
    'email' =&gt; 'anna@example.com',
    'age'   =&gt; 28,
]);
echo $pdo-&gt;lastInsertId();  // id новой записи
```

Чтение одной записи:

```
$stmt = $pdo-&gt;prepare(
    'SELECT * FROM users WHERE id = :id'
);
$stmt-&gt;execute(['id' =&gt; 42]);
$user = $stmt-&gt;fetch();
if ($user === false) {
    echo 'Пользователь не найден';
} else {
    echo $user['name'];
}
```

Чтение всех подходящих записей:

```
$stmt = $pdo-&gt;query(
    'SELECT * FROM users ORDER BY name'
);
foreach ($stmt as $row) {
    echo $row['name'] . '&lt;br&gt;';
}
```

Обновление:

```
$stmt = $pdo-&gt;prepare(
    'UPDATE users SET age = :age WHERE id = :id'
);
$stmt-&gt;execute(['age' =&gt; 29, 'id' =&gt; 42]);
echo $stmt-&gt;rowCount() . ' строк обновлено';
```

Удаление:

```
$stmt = $pdo-&gt;prepare('DELETE FROM users WHERE id = :id');
$stmt-&gt;execute(['id' =&gt; 42]);
```

8.5. Обработка ошибок

В режиме ERRMODE_EXCEPTION любые ошибки SQL бросают PDOException. Оборачивайте опасные операции в try/catch, чтобы обрабатывать ошибки красиво, а не сыпать пользователю технические детали:

```
try {
    $stmt = $pdo-&gt;prepare(
        'INSERT INTO users (email) VALUES (:email)'
    );
    $stmt-&gt;execute(['email' =&gt; $email]);
} catch (PDOException $e) {
    if ($e-&gt;getCode() === '23000') {
        // Нарушение уникального ключа
        echo 'Email уже зарегистрирован';
    } else {
        // Логируем $e в файл, пользователю — общую ошибку
        error_log($e-&gt;getMessage());
        echo 'Внутренняя ошибка сервера';
    }
}
```

ГЛАВА 9. ОСНОВЫ ООП

9.1. Классы и объекты

Класс — это шаблон. Объект — конкретный экземпляр, созданный из шаблона. Аналогия: класс — это чертёж дома, объект — построенный по нему дом. Дома разные, но череп и план у них один.

```
&lt;?php
class User {
    public string $name;
    public int $age;
    private array $emails = [];

    public function __construct(string $name, int $age) {
        $this-&gt;name = $name;
        $this-&gt;age  = $age;
    }

    public function addEmail(string $email): void {
        $this-&gt;emails[] = $email;
    }

    public function getEmails(): array {
        return $this-&gt;emails;
    }
}

$u = new User('Анна', 28);
$u-&gt;addEmail('anna@example.com');
echo $u-&gt;name;  // Анна
print_r($u-&gt;getEmails());
?&gt;
```

Что здесь нового по сравнению с процедурным PHP:

- public/private — модификаторы доступа к свойствам и методам
- $this — ссылка на текущий объект внутри его методов
- __construct — метод, вызываемый при создании объекта (new)
- string $name — типизированное свойство (PHP 7.4+)

9.2. Модификаторы доступа

Видимость свойств и методов:

- public — доступно отовсюду: снаружи, в наследниках, в самом классе
- protected — доступно только внутри класса и его наследников
- private — доступно только внутри самого класса

Правило: всё, что можно скрыть (private), должно быть скрыто. Открывать (public) только то, что действительно является интерфейсом класса — методы, которые вызывает внешний код. Это называется инкапсуляцией и защищает от случайных изменений внутренних данных.

9.3. Наследование

Один класс может расширять другой через ключевое слово extends. Наследник получает все public и protected свойства и методы родителя:

```
class Animal {
    public function breathe(): void {
        echo 'Дышит...';
    }
}

class Dog extends Animal {
    public function bark(): void {
        echo 'Гав-гав!';
    }
}

$rex = new Dog();
$rex-&gt;breathe();  // Дышит... (наследовано)
$rex-&gt;bark();     // Гав-гав!
```

Метод родителя можно переопределить в наследнике. Чтобы вызвать оригинал — parent::method():

```
class Puppy extends Dog {
    public function bark(): void {
        parent::bark();       // Гав-гав!
        echo ' Тяв-тяв!';
    }
}
```

9.4. readonly свойства (PHP 8.1)

В PHP 8.1 появилась возможность объявить свойство readonly. Оно записывается один раз — обычно в конструкторе — и после этого не может быть изменено:

```
class Point {
    public function __construct(
        public readonly float $x,
        public readonly float $y,
    ) {}
}

$p = new Point(1.5, 2.0);
echo $p-&gt;x;  // 1.5
// $p-&gt;x = 3.0;  // Ошибка: readonly
```

Это особый синтаксис constructor promotion: объявление свойств прямо в параметрах конструктора. PHP сам создаёт свойства и присваивает значения. readonly полезен для value-объектов — таких как деньги, координаты, ID, которые не должны меняться после создания.

9.5. Пространства имён (namespace)

Когда проект растёт, имена классов могут сталкиваться. Решение — пространства имён. Они указываются в начале файла и работают как папки в файловой системе:

```
&lt;?php
namespace App\Models;

class User {
    // ...
}
?&gt;

&lt;!-- В другом файле --&gt;
&lt;?php
use App\Models\User;

$u = new User();
?&gt;
```

Соглашение об именовании: имена пространств повторяют структуру папок. Класс App\Models\User лежит в файле app/Models/User.php. Это упрощает автозагрузку через Composer — стандартный менеджер зависимостей PHP.

ГЛАВА 10. БЕЗОПАСНОСТЬ

10.1. SQL-инъекции

Если вы подставляете данные пользователя прямо в SQL-запрос через конкатенацию строк, вы открываете дверь для SQL-инъекций. Классический пример:

```
// ОПАСНО — НИКОГДА ТАК НЕ ДЕЛАЙТЕ
$id = $_GET['id'];
$sql = "SELECT * FROM users WHERE id = $id";
$pdo-&gt;query($sql);
```

Злоумышленник может передать id=1 OR 1=1 и получить весь список пользователей. Или id=1; DROP TABLE users — и потерять таблицу. Защита простая: подготовленные запросы с плейсхолдерами (глава 8). Это 100% защита от SQL-инъекций для данных.

```
$stmt = $pdo-&gt;prepare('SELECT * FROM users WHERE id = ?');
$stmt-&gt;execute([$_GET['id']]);
$user = $stmt-&gt;fetch();
```

10.2. XSS — межсайтовый скриптинг

XSS — это когда пользователь вводит в форму JavaScript-код, и он выполняется в браузере другого пользователя. Например, в комментарий кладут <script>fetch('/delete-account', {method:'POST'})</script>. Когда администратор открывает страницу с комментарием, скрипт выполняется от его имени.

Защита: при выводе любого пользовательского текста в HTML используйте htmlspecialchars:

```
$comment = $_POST['comment'];

// Плохо:
echo $comment;

// Хорошо:
echo htmlspecialchars($comment, ENT_QUOTES, 'UTF-8');
```

Функция заменяет < > " ' & на HTML-сущности (< > " ' &). Браузер отображает их как текст, а не как HTML-теги. Флаг ENT_QUOTES экранирует и одинарные кавычки — это важно, если вывод попадает в атрибут тега.

Современный подход — использовать шаблоны с автоматическим экранированием (Twig, Blade). В простом PHP экранируйте вручную при каждом выводе.

10.3. CSRF — подделка запроса

CSRF (Cross-Site Request Forgery) — когда чужой сайт отправляет форму на ваш сайт от имени авторизованного пользователя. Браузер автоматически прикладывает куки сессии, поэтому сервер думает, что действие делает сам пользователь.

Защита — CSRF-токен: случайная строка, которую сервер кладёт в сессию и в форму. При получении формы сервер проверяет, что токен из формы совпадает с токеном из сессии. Чужой сайт не имеет доступа к сессии и не может подделать токен.

```
&amp;lt;?php
session_start();

// Генерация токена
if (empty($_SESSION['csrf'])) {
    $_SESSION['csrf'] = bin2hex(random_bytes(32));
}
?&amp;gt;

&amp;lt;form method="post"&amp;gt;
  &amp;lt;input type="hidden" name="csrf"
    value="&amp;lt;?= $_SESSION['csrf'] ?&amp;gt;"&amp;gt;
  &amp;lt;input name="email"&amp;gt;
  &amp;lt;button&amp;gt;Подписаться&amp;lt;/button&amp;gt;
&amp;lt;/form&amp;gt;
```

```
&lt;?php
session_start();
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    if (!hash_equals($_SESSION['csrf'],
                     $_POST['csrf'] ?? '')) {
        die('Неверный CSRF-токен');
    }
    // Обработка формы
}
?&gt;
```

Функция hash_equals сравнивает строки за постоянное время — это защищает от timing-атак, когда злоумышленник пытается подобрать токен по времени ответа сервера. Используйте hash_equals вместо === для сравнения любых секретов.

10.4. Хранение паролей

Пароли никогда не хранятся в открытом виде в базе. Даже если утечёт база, хэшированные пароли останутся бесполезными. PHP предоставляет две функции для безопасной работы с паролями:

```
// Хэширование при регистрации
$hash = password_hash($password, PASSWORD_DEFAULT);
// $hash хранится в базе. PASSWORD_DEFAULT
// использует bcrypt + случайную соль.

// Проверка при входе
if (password_verify($input_password, $hash)) {
    // Пароль верный
} else {
    // Пароль неверный
}
```

Несколько важных моментов:

- password_hash сам генерирует соль — не нужно ничего придумывать
- Полученный $hash содержит алгоритм, соль и сам хэш
- PASSWORD_DEFAULT сегодня = bcrypt, завтра может смениться
- password_verify корректно сравнивает при любом алгоритме

Для обновления хэшей, когда алгоритм устарел, есть password_needs_rehash. Вызывайте её после успешного входа — если вернёт true, пересчитайте хэш современным алгоритмом и обновите в базе.

```
if (password_verify($input, $hash)) {
    if (password_needs_rehash($hash, PASSWORD_DEFAULT)) {
        $newHash = password_hash($input, PASSWORD_DEFAULT);
        // UPDATE users SET password = ? WHERE id = ?
    }
    // Логиним пользователя
}
```

10.5. Валидация входных данных

Главное правило: никогда не доверяйте данным от пользователя. Всё, что приходит в $_GET, $_POST, $_COOKIE, $_FILES, потенциально враждебно. Проверяйте:

- Тип: число ли это, дата ли это, email ли это
- Диапазон: 1 <= age <= 120
- Длина: strlen($name) между 2 и 50
- Принадлежность: $role in ['admin','user','guest']
- Формат: телефон, ИНН, номер карты — через filter_var или регулярки

```
$age = filter_input(INPUT_POST, 'age',
    FILTER_VALIDATE_INT,
    ['options' =&gt; ['min_range' =&gt; 1, 'max_range' =&gt; 120]]);
if ($age === false) {
    $errors[] = 'Возраст должен быть от 1 до 120';
}
```

ГЛАВА 11. ПРАКТИЧЕСКИЕ ЗАДАЧИ

11.1. Задача 1: счётчик посетителей

Сделайте страницу, которая считает, сколько раз её открыли. Число хранится в файле counter.txt. Решение:

```
&lt;?php
$file = __DIR__ . '/counter.txt';
$count = (int) file_get_contents($file);
$count++;
file_put_contents($file, $count, LOCK_EX);
?&gt;

&lt;!DOCTYPE html&gt;
&lt;html&gt;&lt;body&gt;
  &lt;h1&gt;Страницу открыли &lt;?= $count ?&gt; раз&lt;/h1&gt;
&lt;/body&gt;&lt;/html&gt;
```

11.2. Задача 2: форма обратной связи

Сделайте форму контактов: имя, email, сообщение. После отправки сохраняйте в файл и показывайте спасибо. Решение:

```
&lt;?php
$errors = [];
$sent = false;

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $name = trim($_POST['name'] ?? '');
    $email = trim($_POST['email'] ?? '');
    $msg = trim($_POST['message'] ?? '');

    if ($name === '') $errors[] = 'Имя обязательно';
    if (!filter_var($email, FILTER_VALIDATE_EMAIL))
        $errors[] = 'Некорректный email';
    if (mb_strlen($msg) &lt; 10)
        $errors[] = 'Сообщение слишком короткое';

    if (!$errors) {
        $line = date('c') . " | $name | $email\n"
             . str_replace("\n", ' ', $msg) . "\n---\n";
        file_put_contents('messages.txt', $line,
            FILE_APPEND | LOCK_EX);
        $sent = true;
    }
}
?&gt;

&lt;?php if ($sent): ?&gt;
  &lt;p&gt;Спасибо за сообщение!&lt;/p&gt;
&lt;?php else: ?&gt;
  &lt;?php foreach ($errors as $e): ?&gt;
    &lt;p style="color:red"&gt;&lt;?= htmlspecialchars($e) ?&gt;&lt;/p&gt;
  &lt;?php endforeach; ?&gt;
  &lt;form method="post"&gt;
    &lt;input name="name" placeholder="Имя"&gt;&lt;br&gt;
    &lt;input name="email" placeholder="Email"&gt;&lt;br&gt;
    &lt;textarea name="message"&gt;&lt;/textarea&gt;&lt;br&gt;
    &lt;button&gt;Отправить&lt;/button&gt;
  &lt;/form&gt;
&lt;?php endif; ?&gt;
```

11.3. Задача 3: чтение CSV

Прочитайте файл users.csv и выведите таблицу с данными. Решение:

```
&lt;?php
$rows = [];
if (($fh = fopen('users.csv', 'r')) !== false) {
    $header = fgetcsv($fh);  // первая строка — заголовки
    while (($row = fgetcsv($fh)) !== false) {
        $rows[] = array_combine($header, $row);
    }
    fclose($fh);
}
?&gt;

&lt;table border="1"&gt;
  &lt;tr&gt;
    &lt;?php foreach ($header as $col): ?&gt;
      &lt;th&gt;&lt;?= htmlspecialchars($col) ?&gt;&lt;/th&gt;
    &lt;?php endforeach; ?&gt;
  &lt;/tr&gt;
  &lt;?php foreach ($rows as $row): ?&gt;
    &lt;tr&gt;
      &lt;?php foreach ($row as $cell): ?&gt;
        &lt;td&gt;&lt;?= htmlspecialchars($cell) ?&gt;&lt;/td&gt;
      &lt;?php endforeach; ?&gt;
    &lt;/tr&gt;
  &lt;?php endforeach; ?&gt;
&lt;/table&gt;
```

11.4. Задача 4: мини-блог с PDO

Выведите список статей из таблицы articles(id, title, body, created_at). Решение:

```
&lt;?php
require_once 'db.php';  // создаёт $pdo

$stmt = $pdo-&gt;query(
    'SELECT * FROM articles ORDER BY created_at DESC'
);
$articles = $stmt-&gt;fetchAll();
?&gt;

&lt;?php foreach ($articles as $a): ?&gt;
  &lt;article&gt;
    &lt;h2&gt;&lt;?= htmlspecialchars($a['title']) ?&gt;&lt;/h2&gt;
    &lt;time&gt;&lt;?= $a['created_at'] ?&gt;&lt;/time&gt;
    &lt;p&gt;&lt;?= nl2br(htmlspecialchars($a['body'])) ?&gt;&lt;/p&gt;
  &lt;/article&gt;
&lt;?php endforeach; ?&gt;
```

Функция nl2br превращает переводы строк в
 — удобно для сохранения абзацев при выводе текста из textarea. Важно: htmlspecialchars вызывается ПЕРЕД nl2br, иначе преобразование
 было бы тоже экранировано.

11.5. Задача 5: проверка пароля

Напишите функцию, которая принимает пароль и хэш, возвращает true/false и при успехе обновляет хэш, если он устарел. Решение:

```
function check_password(
    string $password,
    string $hash
): bool {
    if (!password_verify($password, $hash)) {
        return false;
    }
    if (password_needs_rehash($hash, PASSWORD_DEFAULT)) {
        $new = password_hash($password, PASSWORD_DEFAULT);
        // TODO: сохранить $new в базу вместо $hash
    }
    return true;
}
```

11.6. Задача 6: JSON API

Сделайте endpoint api.php, который возвращает список пользователей в JSON. Решение:

```
&lt;?php
header('Content-Type: application/json; charset=utf-8');
require_once 'db.php';

$stmt = $pdo-&gt;query('SELECT id, name, email FROM users');
$users = $stmt-&gt;fetchAll();

echo json_encode($users, JSON_UNESCAPED_UNICODE);
?&gt;
```

Флаг JSON_UNESCAPED_UNICODE отключает экейп кириллицы в \u0410-подобные последовательности — без него JSON выглядит как \u0410\u043d\u043d\u0430 вместо читаемого текста. Полезно для отладки и небольших API.

11.7. Задача 7: класс User

Создайте класс User с readonly-свойствами name и email, методом greeting(). Решение:

```
&lt;?php
class User {
    public function __construct(
        public readonly string $name,
        public readonly string $email,
    ) {}

    public function greeting(): string {
        return "Привет, я {$this-&gt;name}!";
    }
}

$u = new User('Анна', 'anna@example.com');
echo $u-&gt;greeting();  // Привет, я Анна!
echo $u-&gt;email;       // anna@example.com
// $u-&gt;name = 'Олег';  // Ошибка: readonly
?&gt;
```

ГЛАВА 12. ИТОГИ И СПРАВОЧНИК ФУНКЦИЙ

12.1. Итоги по главам

Глава 1. PHP — серверный язык. Код выполняется на сервере, пользователю отдаётся готовый HTML. Локальный сервер: php -S localhost:8000.

Глава 2. Переменные начинаются с $. PHP динамически типизирован, но в функциях можно указывать типы. Условия if/elseif/else, match в PHP 8.x, циклы while/for/foreach.

Глава 3. Функции объявляются через function. Аргументы по умолчанию, именованные аргументы (PHP 8), типизация параметров и возврата. include/require для подключения файлов.

Глава 4. Массивы бывают индексные и ассоциативные. foreach перебирает оба. Функции: count, array_map, array_filter, array_merge, sort. Spread-оператор ... распаковывает массивы.

Глава 5. Суперглобалы $_GET, $_POST, $_SERVER, $_FILES. GET — для чтения, POST — для изменения. Никогда не доверяйте данным из формы, валидируйте всё.

Глава 6. file_get_contents/file_put_contents для простых случаев. fopen/fgets для больших файлов. Загрузка файлов через $_FILES + move_uploaded_file, обязательно проверять MIME на сервере.

Глава 7. Куки хранятся в браузере, сессии — на сервере. session_start() включает сессию, $_SESSION — для данных. setcookie — для кук, обязательно до вывода.

Глава 8. PDO — стандарт работы с БД. Подготовленные запросы с плейсхолдерами — защита от SQL-инъекций. CRUD через prepare/execute. Режим ERRMODE_EXCEPTION обязателен.

Глава 9. Классы и объекты: public/protected/private, __construct, наследование через extends. readonly (PHP 8.1). Namespace для организации кода по папкам.

Глава 10. SQL-инъекции — защищаемся подготовленными запросами. XSS — htmlspecialchars при выводе. CSRF — токены в форме и в сессии. Пароли — password_hash/password_verify.

Глава 11. Семь практических задач показали, как combine theory на практике: счётчик, форма, CSV, мини-блог, проверка пароля, JSON API, класс User.

12.2. Справочник частых функций

Строки:

- strlen($s) — длина в байтах (для UTF-8 — mb_strlen)
- mb_strtolower($s), mb_strtoupper($s) — регистр
- trim($s), ltrim, rtrim — обрезка пробелов
- str_replace($from, $to, $s) — замена
- str_contains($s, $sub) — содержит (PHP 8)
- str_starts_with / str_ends_with (PHP 8)
- substr($s, $start, $len) — подстрока
- explode($sep, $s) — разбить в массив
- implode($sep, $arr) — собрать из массива
- htmlspecialchars($s) — экранирование HTML

Массивы:

- count($arr) — размер
- in_array($v, $arr) — поиск значения
- array_keys / array_values — ключи / значения
- array_merge($a, $b) — объединить
- array_map($fn, $arr) — применить функцию
- array_filter($arr, $fn) — отфильтровать
- array_reverse($arr) — перевернуть
- sort($arr) / rsort / asort / arsort — сортировки

- usort($arr, $fn) — пользовательская сортировка
- array_slice($arr, $offset, $len) — срез

Математика:

- abs($n), max(...$nums), min(...$nums)
- round($n, $precision), ceil($n), floor($n)
- rand($min, $max), random_int($min, $max) — криптостойкое
- random_bytes($len) — случайные байты
- bin2hex($bytes) — байты в HEX-строку

Даты:

- time() — Unix-время в секундах
- date('d.m.Y') — отформатировать текущую дату
- date('d.m.Y', $ts) — отформатировать произвольную
- strtotime('next Monday') — парсит строку в timestamp
- mktime($h,$m,$s,$mo,$d,$y) — собрать timestamp из частей

Файлы:

- file_get_contents($path) — прочитать файл целиком
- file_put_contents($path, $data, $flags) — записать
- file($path) — массив строк файла
- fopen/fgets/fwrite/fclose — потоковый доступ
- is_file / is_dir / file_exists — проверки
- unlink($path) — удалить файл
- rename($old, $new) — переименовать

JSON:

- json_encode($data, JSON_UNESCAPED_UNICODE)
- json_decode($json, true) — true = ассоциативный массив
- json_last_error() / json_last_error_msg() — ошибки

Регулярные выражения:

- preg_match($pattern, $subject, &$matches)
- preg_match_all($pattern, $subject, &$matches)
- preg_replace($pattern, $replacement, $subject)
- preg_split($pattern, $subject)

Что читать дальше

После освоения этого учебника логично двигаться в трёх направлениях. Во-первых, Composer — стандартный менеджер зависимостей PHP, без которого не обходится ни один современный проект. Во-вторых, фреймворки: Laravel (полный стек, быстрый старт), Symfony (компоненты, гибкость), Slim (минимализм). В-третьих, тестирование: PHPUnit, Pest.

Главный совет: не пытайтесь заучить все функции. PHP — это практика. Напишите свой мини-блог с авторизацией, гостевую книгу, список задач с категориями. С каждой новой строкой кода приходит интуитивное понимание, когда нужна сессия, а когда кука, когда prepare, а когда query, когда класс, а когда функция. Опыт складывается из десятков сделанных и переделанных проектов, а не из прочитанных справочников.
